SENAI

Serviço Nacional de Aprendizagem Industrial

Habilitação

TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

Módulo

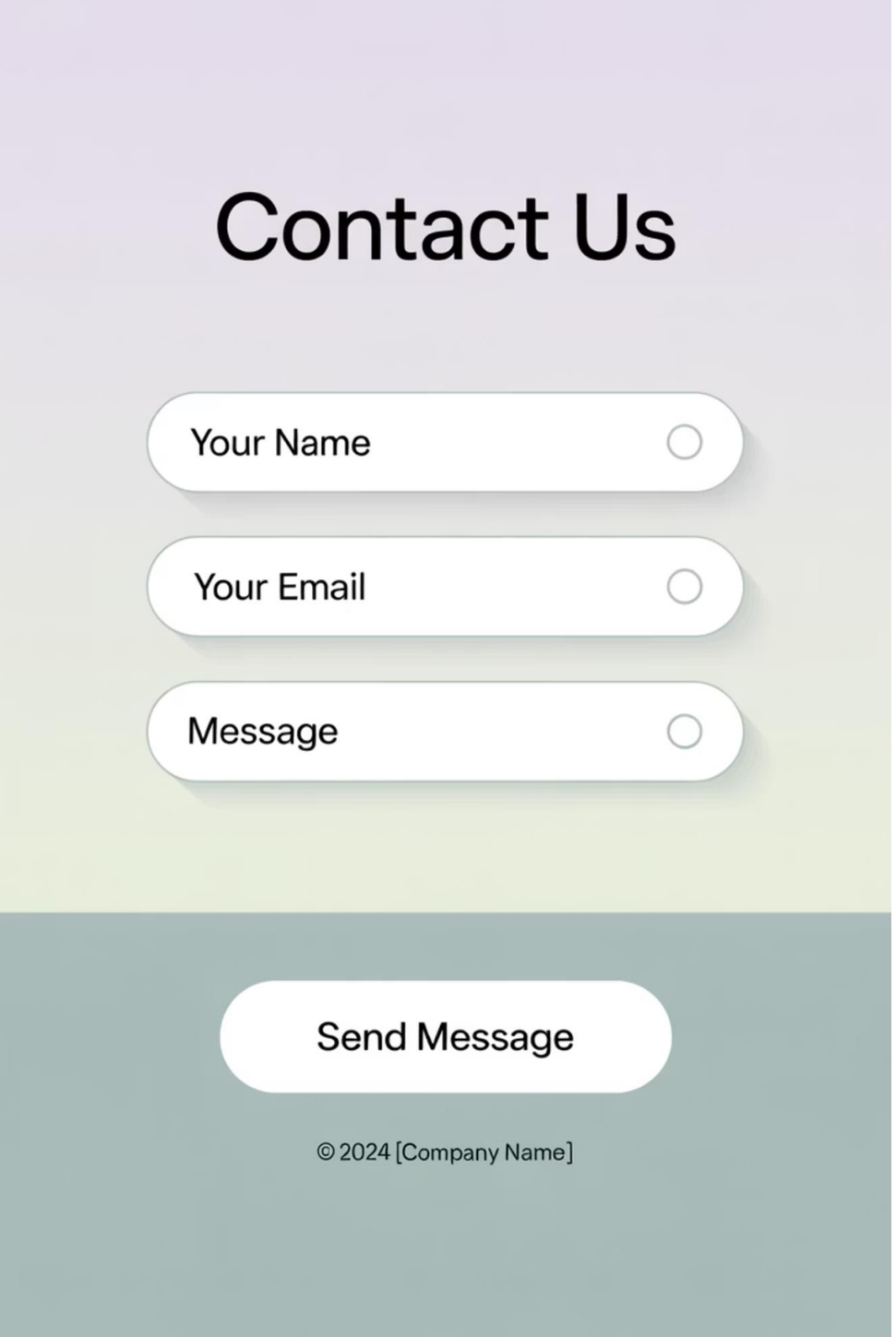
MÓDULO ESPECÍFICO I

Unidade Curricular

LINGUAGEM DE MARCAÇÃO

Instrutor

Leonardo Evangelista



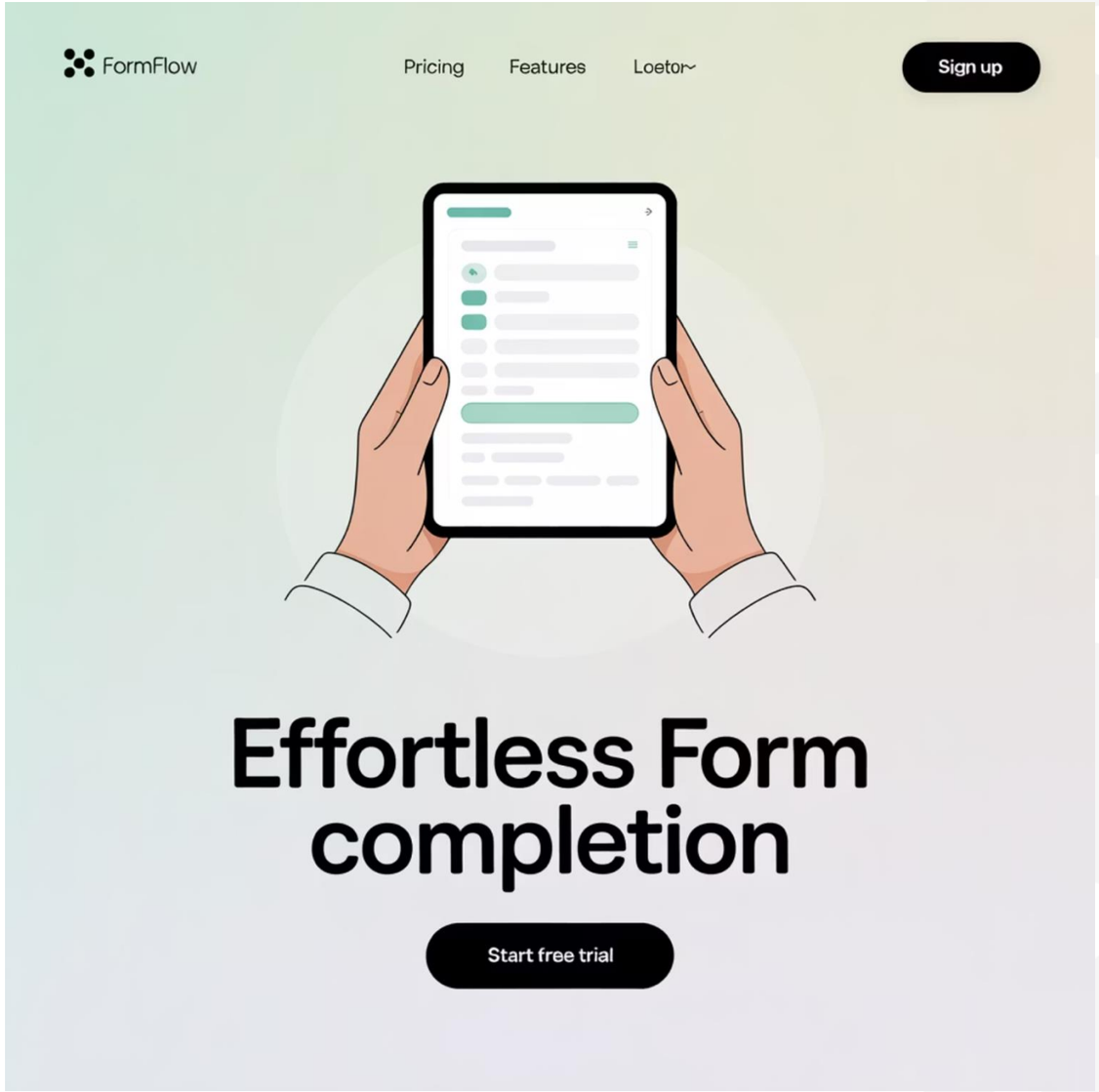
SENAI

Formulários HTML

O que são Formulários HTML?

Formulários HTML são elementos essenciais das páginas web que permitem a interação entre usuários e sistemas. Eles funcionam como pontes digitais, coletando informações dos visitantes e enviando-as para processamento.

Imagine um formulário como uma ficha de cadastro digital - assim como preenchemos formulários em papel no mundo real, os formulários HTML servem ao mesmo propósito no mundo digital. Eles capturam dados como nomes, emails, senhas e muito mais.



Por que Formulários são Importantes?



Cadastro de Usuários

Permitem que novos usuários se registrem em sites, criando contas pessoais com informações básicas como nome, email e senha.



Compras Online

Essenciais para e-commerce, coletando dados de pagamento, endereço de entrega e informações do cartão de crédito.



Contato e Feedback

Facilitam a comunicação entre visitantes e empresas através de formulários de contato, pesquisas e avaliações.



Busca e Filtros

Permitem pesquisas personalizadas, filtros de produtos e refinamento de resultados conforme as necessidades do usuário.

Estrutura Básica de um Formulário

Todo formulário HTML segue uma estrutura fundamental que você precisa conhecer:

01

Tag de Abertura <form>

Inicia o formulário e define suas configurações principais, como destino dos dados e método de envio.

02

Campos de Entrada

Elementos como input, textarea e select onde os usuários inserem suas informações.

03

Botão de Envio

Elemento que permite ao usuário submeter os dados preenchidos para processamento.

04

Tag de Fechamento </form>

Finaliza o formulário, definindo onde termina a área de coleta de dados.

```html
<!DOCTYPE html>
<html lang="pt-BR">
    <head>
        <meta charset="UTF-8">
        <title>Listas</title>
    </head>
    <body>
        <form action="/processar" method="POST">
            <input type="text" name="nome" placeholder="Seu nome">
            <input type="email" name="email" placeholder="Seu email">
            <button type="submit">Cadastrar</button>
        </form>
    </body>
```



Tipos de Campos de Entrada - <input >

HTML oferece diversos tipos de campos para diferentes necessidades:

type="text"

Campo básico para texto simples como nomes, títulos ou descrições curtas.

type="email"

Específico para endereços de email, com validação automática do formato.

type="password"

Oculta o texto digitado, ideal para senhas e informações confidenciais.

type="number"

Aceita apenas números, perfeito para idades, quantidades e valores numéricos.

type= "date"

Abre um seletor de calendário, facilitando a escolha de datas de nascimento, eventos ou prazos.

type="checkbox "

Permite múltiplas seleções, como aceitar termos, escolher interesses ou preferências.

type="radio"

Oferece opções exclusivas onde apenas uma pode ser selecionada, como gênero ou tamanho.

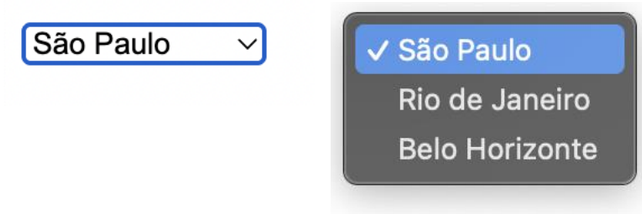
Elementos Especiais: Select e Textarea

Lista Suspensa (Select) - Dropdown

O elemento <select> cria listas de opções organizadas:

```
<select name="cidade">
    <option>São Paulo</option>
    <option>Rio de Janeiro</option>
    <option>Belo Horizonte</option>
</select>
```

Ideal quando há muitas opções para escolher, economizando espaço na tela.

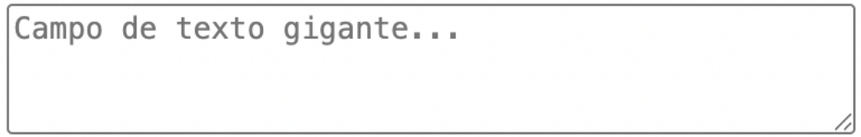


Área de Texto (Textarea)

Para textos longos como comentários ou descrições:

```
<textarea name="mensagem" rows="4" cols="5" placeholder="Campo de texto gigante...">
</textarea>
```

Permite ao usuário escrever textos extensos com quebras de linha e formatação básica.



Atributos Importantes dos Campos

name="campo"

Identifica o campo quando os dados são enviados. É como um rótulo único para cada informação coletada.

placeholder="texto"

Mostra uma dica visual dentro do campo vazio, orientando o usuário sobre o que digitar.

required

Torna o preenchimento obrigatório. O formulário não será enviado se campos obrigatórios estiverem vazios.

value="padrão"

Define um valor inicial para o campo, útil para sugerir respostas ou pré-preencher informações.

O formulário precisa saber para onde enviar

```
<form action="processa.php" method="post">
```

action

Indica **para qual arquivo PHP** os dados serão enviados quando o usuário clicar em Enviar. No exemplo acima, os dados vão para `processa.php`.

method

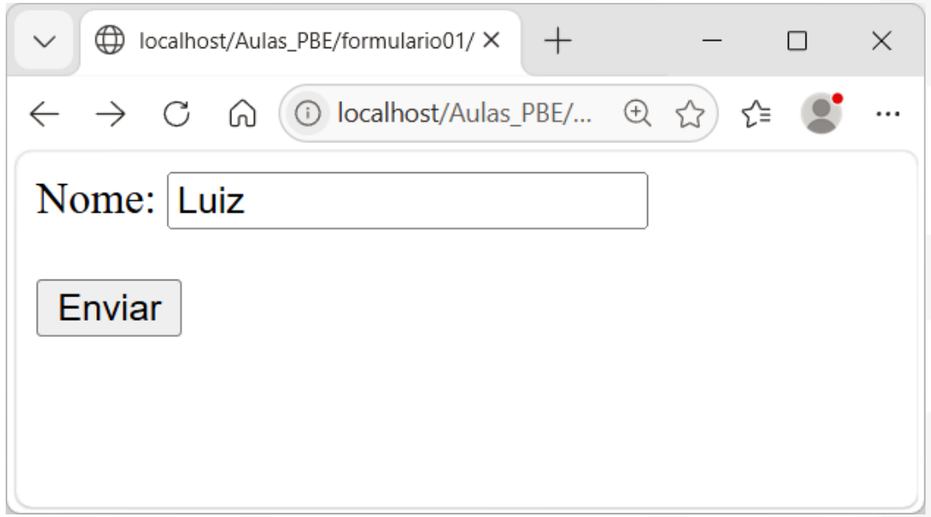
Indica **como** os dados serão enviados. Vamos trabalhar principalmente com `method="post"`, que envia os dados de forma segura no corpo da requisição, sem exibi-los na URL.

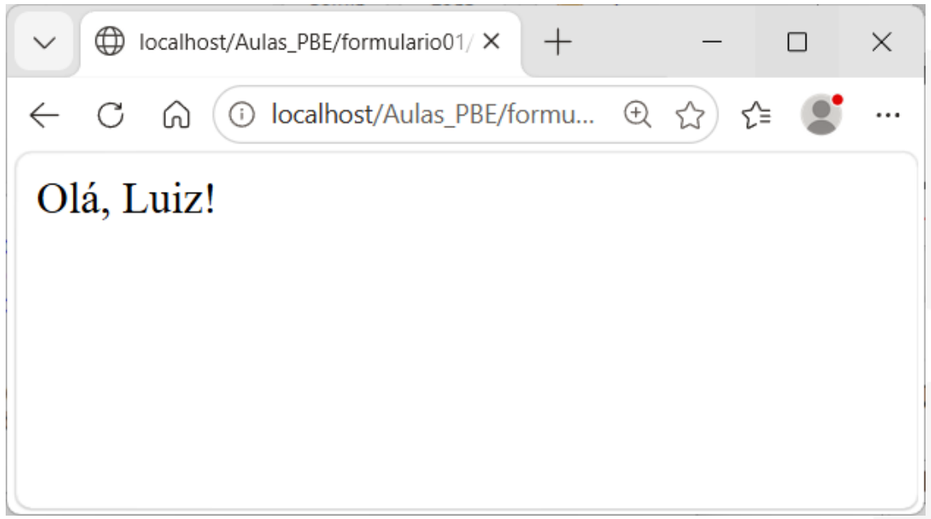
```
1  <form action="processa.php" method="post">
2      <label>Nome:</label>
3      <input type="text" name="nome">
4      <br><br>
5      <button type="submit">Enviar</button>
6  </form>
```

Criando o processa.php

processa.php — O processamento

```php
<?php
$nome = $_POST["nome"];
echo "Olá, $nome!";
```





Como o PHP recebe os dados?

Quando usamos `method="post"`**...**

O PHP disponibiliza automaticamente uma variável especial chamada **$_POST**. Ela contém todos os dados enviados pelo formulário.

```
$nome = $_POST["nome"];
```

Após essa linha, a variável `$nome` contém exatamente o que o usuário digitou no campo com `name="nome"`.

Visualizando a ligação

1

Formulário envia

`name="nome"` identifica o campo

2

PHP captura

`$_POST["nome"]` lê o valor

3

Variável armazena

`$nome` fica pronta para uso

O atributo `name`: o elo entre HTML e PHP

O atributo `name` em um campo de formulário é **fundamental**. Sem ele, o PHP não consegue identificar o dado enviado.

No HTML

```
<input type="text" name="idade">
```

O atributo `name="idade"` nomeia o campo. É como dar uma etiqueta ao dado.

No PHP

```
$_POST["idade"]
```

O PHP usa exatamente esse nome para localizar o valor recebido.

O `name` do HTML e o índice usado no PHP **precisam ser idênticos**. Uma diferença, mesmo que pequena, fará o PHP não encontrar o dado.

$_POST é um array

A variável `$_POST` funciona como um **array associativo**: cada chave corresponde ao `name` de um campo do formulário, e o valor é o que o usuário digitou.

No HTML

```
<input type="text" name="nome">
<input type="number" name="idade">
```

Dois campos com `name` distintos.

No PHP

```
$_POST["nome"]   // valor do campo nome
$_POST["idade"]  // valor do campo idade
```

Cada `name` do formulário vira uma chave em `$_POST`.

Se o formulário tiver 5 campos com `name` diferentes, o `$_POST` terá 5 entradas, uma para cada campo.



Métodos HTTP

Como os formulários são enviados pela internet?

O que são Métodos HTTP?

Métodos HTTP são formas padronizadas de comunicação entre o navegador (cliente) e o servidor. Eles definem como as informações serão transmitidas quando você clica em "Enviar" em um formulário.

1

Navegador

Usuário preenche o formulário e clica em enviar

2

Método HTTP

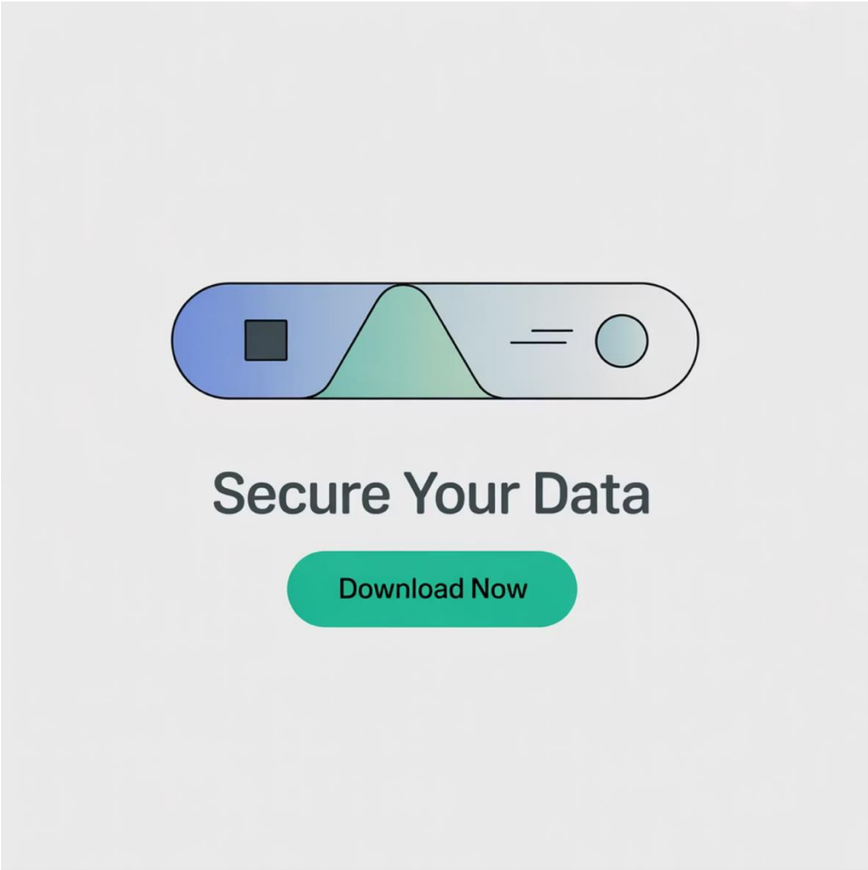
Define como os dados serão empacotados e enviados

3

Servidor

Recebe e processa as informações conforme programado

Método GET: Como Funciona



O método GET envia dados através da URL, tornando-os visíveis na barra de endereços do navegador.

Características principais:

- Dados aparecem na URL após o símbolo "?"
- Limitado a aproximadamente 2000 caracteres
- Informações ficam visíveis no histórico do navegador
- Pode ser marcado como favorito com os dados incluídos

Exemplo Prático do GET

Quando você usa um formulário GET, os dados ficam expostos na URL:

```
<form method="GET" action="/buscar">
    <input type="text" name="produto" placeholder="O que procura?">
    <input type="text" name="categoria" placeholder="Categoria">
    <button type="submit">Buscar</button>
</form>
```

O que procura? Categoria Buscar

Após o enviu

notebook informática Buscar

A URL ficaria assim:

/buscar?produto=notebook&categoria=informática

Note como os dados do formulário aparecem claramente após o "?" na URL.

Quando Usar o Método GET

Buscas e Pesquisas

Perfeito para campos de busca, onde o usuário pode querer compartilhar ou repetir a mesma pesquisa.

Filtros de Produtos

Ideais para filtrar resultados por preço, categoria ou características, permitindo favoritar a página filtrada.

URLs Compartilháveis

Quando você quer que o usuário possa compartilhar o link com os dados preenchidos.



Protect

Método POST: Segurança em Primeiro Lugar

O método POST envia dados de forma oculta, através do corpo da requisição HTTP, não aparecendo na URL.

Dados Ocultos

As informações não aparecem na barra de endereços, mantendo privacidade.

Sem Limite de Tamanho

Pode enviar grandes quantidades de dados, incluindo arquivos e textos longos.

Mais Seguro

Senhas e dados confidenciais não ficam expostos no histórico do navegador.

Exemplo Prático do POST

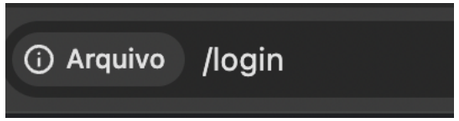
```
<form method="POST" action="/login">
    <input type="email" name="email" placeholder="Seu email" required>
    <input type="password" name="senha" placeholder="Sua senha" required>
    <button type="submit">Entrar</button>
</form>
```



Após o enviu



A URL ficaria assim:



Os dados (email e senha) são enviados de forma oculta, protegendo informações sensíveis. Esta é a forma correta de enviar dados confidenciais.

Quando Usar o Método POST

Login e Senhas

Sempre use POST para formulários que contenham senhas ou dados de autenticação.



Cadastros Pessoais



Para registros de usuários com informações pessoais e confidenciais.

Upload de Arquivos

Necessário para enviar arquivos, imagens ou documentos.



Pagamentos



Fundamental para transações financeiras e dados de cartão de crédito.

Comparação: GET vs POST

Aspecto	GET	POST
Visibilidade dos dados	Visível na URL	Oculto no corpo da requisição
Limite de dados	~2000 caracteres	Sem limite prático
Segurança	Menos seguro	Mais seguro
Cache do navegador	Pode ser armazenado	Não é armazenado
Uso principal	Buscas e filtros	Envio de dados sensíveis

Formulário Completo: Exemplo Final

Vamos criar um formulário de cadastro que combina tudo que aprendemos:

```html
<form action="cadastrar.php" method="POST">
    <h2>Cadastre-se</h2>

    <input type="text" name="nome" placeholder="Nome completo" required>
    <br><br>

    <input type="email" name="email" placeholder="Seu melhor email" required>
    <br><br>

    <input type="password" name="senha" placeholder="Crie uma senha" required>
    <br><br>

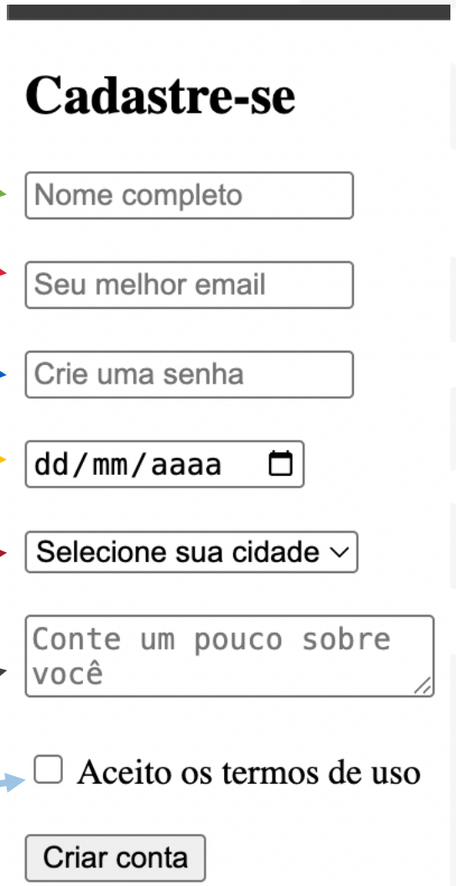
    <input type="date" name="nascimento" required>
    <br><br>

    <select name="cidade" required>
        <option value="">Selecione sua cidade</option>
        <option value="sp">São Paulo</option>
        <option value="rj">Rio de Janeiro</option>
    </select>
    <br><br>

    <textarea name="apresentacao" placeholder="Conte um pouco sobre você"></textarea>
    <br><br>

    <input type="checkbox" name="termos" required>
    <label>Aceito os termos de uso</label>
    <br><br>

    <button type="submit">Criar conta</button>
</form>
```



Formulário Completo: Exemplo Final

Vamos criar um formulário de cadastro que combina tudo que aprendemos:

```php
<?php

$nome = $_POST['nome'];
$email = $_POST['email'];
$senha = $_POST['senha'];
$nascimento = $_POST['nascimento'];
$cidade = $_POST['cidade'];
$apresentacao = $_POST['apresentacao'];

echo "Esse é o valor do nome: $nome";
echo "<br>";

echo "Esse é o valor do email: $email";
echo "<br>";

echo "Essa é a senha: $senha";
echo "<br>";

echo "Essa é a data de nascimento: $nascimento";
echo "<br>";

echo "Essa é a cidade: $cidade";
echo "<br>";

echo "Essa é a apresentação: $apresentacao";
echo "<br>";

 // valor é um bolean true ou false
if (isset($_POST['termos'])) {
    echo "O usuário aceitou os termos de uso.";
}
?>
```

Esse é o valor do nome: leonardo
Esse é o valor do email: leonardo@email.com
Essa é a senha: 1231231
Essa é a data de nascimento: 2026-08-10
Essa é a cidade: sp
Essa é a apresentação: Texto no campo de texto
O usuário aceitou os termos de uso.